NEGROS ORIENTAL STATE UNIVERSITY

ITS 200 – Database Management System
ITS 201 – System Integration and Architecture

FINAL PROJECT DOCUMENTATION

Border Entry Payment and Personnel Tracking System

Group 3
BSINT 2nd Year – 1st Semester

Submitted by:
Dave Cordova
Kenn Caldoza
Kyfer Cainoy
Devine Cabanag
Albert Egay
Michael Fonollera

Submitted to:
Christian Franc M. Carvajal
Instructor

Table of Contents

1. Introduction and Background

2. Problem Statement

3. General and Specific Objectives

4. Scope and Limitations

5. System Requirements

6. Technologies Used

7. System Architecture

8. Database Design and ERD

9. System Implementation and Modules

10. CRUD Operations

11. SQL Queries and JOINs

12. Testing Procedures and Results

13. GitHub Collaboration

14. Conclusion and Recommendations

15. References

16. Appendices

1. Introduction and Background

The Border Entry Payment and Personnel Tracking System is a database-driven information system designed to organize records associated with border entry operations. It brings together information about travelers, vehicles, entry records, checkpoints, lanes, personnel assignments, fee rules, payments, penalties, violations, users, and system activity in a centralized MySQL database.

The project provides a web-based interface for managing operational records and a separate REST API for accessing selected data. The PHP application supports the system's database-oriented management functions, while the Node.js and Express API exposes endpoints that return and process data in JSON format. Both application components connect to the same MySQL database, allowing them to work with a shared set of records.

The intended users include authorized system administrators and personnel responsible for maintaining traveler, vehicle, checkpoint, staffing, fee, payment, and incident-related records. The system is an academic prototype demonstrating database management and system integration concepts; it should not be interpreted as a deployed government border-control platform.

2. Problem Statement

Border entry operations involve several related types of information. If traveler details, vehicle information, entry activity, staff assignments, fees, payments, and violations are maintained separately or manually, users may experience duplicated records, inconsistent information, slow retrieval, and difficulty preparing operational reports.

The project addresses the need for a centralized database and an integrated application interface where related records can be stored, retrieved, updated, and reviewed. It also demonstrates how a web application and a REST API can access a common database.

The project does not establish that the prototype replaces official border inspection, identity verification, legal enforcement, or government payment systems. Those functions require operational, legal, security, and integration requirements beyond the scope of this academic implementation.

3. General and Specific Objectives

General Objective

To design and implement a database-driven Border Entry Payment and Personnel Tracking System that organizes border-entry-related records and demonstrates integration between a PHP web application, a Node.js/Express REST API, and a MySQL database.

Specific Objectives

- To create a relational MySQL database for storing traveler, vehicle, entry, checkpoint, personnel, fee, payment, violation, and user-related information.
- To provide web-based pages for viewing and managing the system's operational records.
- To implement selected REST API endpoints that support data retrieval and traveler record creation, updating, and deletion.
- To define relationships between database tables and use SQL queries and JOIN operations to retrieve related information.
- To organize system information into modules that correspond to the responsibilities of the group members.
- To test the application pages, database structures, SQL JOIN queries, and available API operations and document the observed results.

4. Scope and Limitations

Scope

- Management and storage of traveler and vehicle information.
- Recording and viewing entry records associated with travelers, vehicles, entry types, checkpoints, and lanes.
- Organization of checkpoints, lanes, and vehicle types.
- Storage of fee rules, payment records, penalties, and violation information.
- Recording personnel, departments, shifts, and checkpoint assignments.
- Management of application users and roles.
- Storage of alerts and audit-log records.
- Use of a PHP web application and a Node.js/Express REST API connected to one MySQL database.
- Execution of SQL queries and JOINs to display information from related tables.

Limitations

- The project is an academic prototype and is not confirmed as connected to an official border authority or live government system.
- No live biometric, passport-reader, immigration, customs, or law-enforcement integration has been documented.
- No live online payment gateway or bank transaction integration has been documented. The database contains payment-related records, but this should not be described as actual electronic payment processing.
- The API testing evidence confirms selected GET endpoints and POST, PUT, and DELETE operations for travelers. It does not establish that every API resource supports all four CRUD operations.
- Testing was performed in a local development environment. Production hosting, load testing, penetration testing, disaster recovery, and formal security certification are not documented.

- Actual fee calculations, legal penalty rules, and operational procedures depend on the data and rules entered into the system; the project does not independently validate government policy.

5. System Requirements

The following are practical development and demonstration requirements based on the technologies used. They are recommended baseline requirements, not measured minimum specifications.

Category	Requirement / Purpose
Hardware	A desktop or laptop computer; recommended 4 GB RAM or more; sufficient storage for XAMPP, Node.js, project files, and the database.
Operating system	Windows environment used during development.
Web server / PHP	XAMPP with PHP 8.2.12, or a compatible PHP environment.
Database	MySQL server managed through phpMyAdmin or another MySQL client.
Backend runtime	Node.js (the development environment used Node.js v24.15.0) and npm.
Frontend	A modern web browser; HTML, CSS, and JavaScript files.
Development tools	Visual Studio Code or another code editor; PowerShell or terminal.
API testing	Browser for GET response inspection and PowerShell Invoke-RestMethod for the documented POST, PUT, and DELETE tests. Postman may also be used if configured.
Version control	Git and a GitHub repository for source-code collaboration.

The local development setup used a PHP server at http://127.0.0.1:8000 for the frontend and a Node.js API server on port 3000. These addresses are local development endpoints and are not public deployment addresses.

6. Technologies Used

Technology	Meaning	Purpose in the Project
PHP	A server-side scripting language.	Runs the PHP application pages and performs database-oriented application operations.
Node.js	A JavaScript runtime.	Runs the separate backend API

		server.
Express.js	A web framework for Node.js.	Defines API routes and handles HTTP requests and responses.
MySQL	A relational database management system.	Stores the system's structured records in related tables.
phpMyAdmin	A browser-based MySQL administration tool.	Used to inspect tables, view structures, execute SQL, and create database screenshots.
HTML	HyperText Markup Language.	Defines the structure and content of web pages.
CSS	Cascading Style Sheets.	Controls page layout, appearance, and visual presentation.
JavaScript	A programming language commonly used in browsers and on Node.js.	Supports client-side behavior and backend API implementation.
Git	A distributed version control system.	Tracks source-code changes and supports branch-based development.
GitHub	A platform for hosting Git repositories and collaborating.	Hosts the group repository and supports collaboration through branches and pull requests.
PowerShell	A Windows command-line shell.	Used to start local servers and execute API test requests.
Postman	An API development and testing application.	A suitable tool for sending HTTP requests; the recorded POST/PUT/DELETE evidence in this project was produced using PowerShell rather than Postman.

The PHP interface and the Express API are separate application components. They share the same MySQL database but are not the same server process.

7. System Architecture

The system follows a client–application–database arrangement with two application paths: the PHP web application and the Node.js/Express REST API.

1. User interaction: An authorized user opens the web application through a browser and works with the available system pages.
2. PHP application: The PHP application handles its page requests and database operations using the project's MySQL connection.
3. REST API: Separate clients or testing tools send HTTP requests to the Node.js/Express server. The API processes the route and returns JSON responses.

4. MySQL database: Both application components use the border_entry_system database to read or modify shared records.
5. Response: The relevant application returns a rendered page or JSON response to the requesting client.

During local development, the PHP server was run at 127.0.0.1:8000 and the Express API at localhost:3000. The backend terminal reported that the server was running on port 3000 and that the MySQL connection succeeded.

Insert or retain the project's System_Architecture.png diagram here in the final project folder. The diagram should reflect only components that are actually implemented. External email, payment-gateway, or security services should be labelled as proposed/optional unless the project truly integrates them.

8. Database Design and ERD

The database is named border_entry_system and uses a relational design. A relational database stores information in tables and connects related records through keys. A primary key uniquely identifies a row. A foreign key links a row to a related row in another table and helps maintain referential integrity.

The project database contains the following documented tables:

Table	Meaning / Role
travelers	Stores traveler/person information associated with entry activity.
vehicles	Stores vehicle information associated with travelers and entry records.
entry_records	Records entry-related transactions or events.
checkpoints	Stores checkpoint locations or checkpoint records.
lanes	Stores lane information associated with checkpoints.
vehicle_types	Classifies vehicles by type.
entry_types	Classifies the type of entry record.
personnel	Stores personnel or staff information.
departments	Organizes personnel into departments.
shifts	Stores work-shift information.
checkpoint_assignments	Associates personnel with checkpoint assignments.
users	Stores application user accounts.
roles	Defines user-role categories or access classifications.
fee_rules	Stores fee rule records used by the system.
payments	Stores payment-related records.
penalties	Stores penalty-related records.

violations	Stores violation records.
alerts	Stores alert records associated with system events or violations.
audit_logs	Stores records intended to document system activity.
borderstations	Stores border-station information.

Relationships connect related data and reduce unnecessary duplication. Examples reflected in the documented JOIN work include travelers with entry records, vehicles with travelers, users with roles, personnel with departments, personnel with checkpoint assignments, lanes with checkpoints, payments with fee rules, vehicle types with fee rules, violations with alerts, and audit logs with users.

An Entity Relationship Diagram (ERD) is a visual representation of database entities (tables), their attributes (columns), and relationships. The project's ERD is saved as documentation/ERD.png. The table-structure screenshots provide supporting evidence of the database schema.

9. System Implementation and Modules

The application is divided into functional modules. The following descriptions explain the purpose of each group of tables and the role they play in the overall system. The exact validation rules and all individual screen behaviors should be confirmed against the running application.

9.1 Travelers, Vehicles, and Entry Records — Devine

Purpose: Maintain core information about people, vehicles, and their recorded entry activity.

- Travelers: stores identifying and contact details used by the application. A traveler may be associated with vehicle and entry records.
- Vehicles: stores vehicle information and links vehicles to traveler information where defined by the schema.
- Entry records: stores recorded entry events and connects relevant traveler, vehicle, entry-type, checkpoint, or lane information where those relationships are configured.
- Evidence: application screenshots exist for the Travelers, Vehicles, and Entry Records pages, including Create, Update, and Delete examples for these three modules. API tests also demonstrate traveler creation, update, and deletion.

9.2 Checkpoints, Lanes, and Vehicle Types — Kyfer

Purpose: Organize the physical or logical points and classifications used when recording entry activity.

- Checkpoints: represent checkpoint records used by the entry process.

- Lanes: represent lanes associated with checkpoints, allowing lane information to be stored separately.
- Vehicle types: classify vehicles and can be related to fee rules or other operational records.
- Evidence: database table structures and JOIN-query screenshots document these entities and their relationships. Do not claim live lane sensors or automated vehicle recognition unless separately implemented.

9.3 Fee Rules, Penalties, and Payments — Kenn

Purpose: Keep fee-related and penalty-related information connected to the system's entry and violation records.

- Fee rules: stores fee definitions or rules used by the application.
- Payments: stores payment-related records and can be joined with fee-rule information.
- Penalties: stores penalty-related records associated with the system's enforcement or violation data.
- Evidence: database structures and JOIN results include payments with fee rules, fee rules with entry types, vehicle types with fee rules, and penalty-related records. A live payment gateway has not been documented.

9.4 Personnel, Shifts, and Checkpoint Assignments — Dave

Purpose: Organize staff records and the work schedules or checkpoint assignments recorded by the system.

- Personnel: stores staff-related information.
- Departments: groups personnel by department.
- Shifts: stores shift information.
- Checkpoint assignments: links personnel to checkpoint assignment records.
- Evidence: database structures and JOIN screenshots include departments with personnel and personnel with checkpoint assignments. The available documentation does not establish automated attendance, payroll, or biometric timekeeping.

9.5 Users and Roles — Albert

Purpose: Maintain application accounts and role classifications.

- Users: stores accounts used to access the application, according to the implemented account design.
- Roles: stores role categories that can be associated with users.
- Evidence: a Users and Roles JOIN screenshot and a Users application page are present. Specific permission rules should be described only as implemented and verified in the code.

9.6 Violations, Alerts, and Audit Logs — Michael

Purpose: Store records related to violations, alerts, and traceability of system activity.

- Violations: stores violation-related records.
- Alerts: stores alert records that can be associated with violations or checkpoints.
- Audit logs: stores activity records and can be associated with user accounts.
- Evidence: GET API response screenshots are present for alerts and audit logs, along with JOIN screenshots for violations with alerts, alerts with checkpoints, and audit logs with users. Automatic notifications or immutable audit protection have not been established by the available evidence.

10. CRUD Operations

CRUD is an abbreviation for the four basic operations used to manage stored data:

Operation	Meaning	Description
Create	Add a new record.	A user or API request submits information that is stored as a new database row.
Read	Retrieve or view records.	The application displays stored information or an API returns it to a client.
Update	Modify an existing record.	A user or API request changes selected values in an existing row.
Delete	Remove a record.	A user or API request deletes a selected row, subject to database constraints.

The project's screenshots show Create, Update, and Delete examples for Travelers, Vehicles, and Entry Records. The API test evidence confirms GET responses for selected resources and a successful POST, PUT, and DELETE sequence for a traveler record. The tested traveler record was created with ID 6, updated, and then deleted.

The evidence does not prove that every table or every module supports all four operations through both the PHP interface and the API. CRUD availability should therefore be reported per module based on the actual screen and endpoint implementation.

11. SQL Queries and JOINs

SQL (Structured Query Language) is used to define, retrieve, and manipulate relational database information. A JOIN combines rows from two or more tables using related columns. JOIN queries are useful when information is distributed across related tables.

The project's JOIN-query evidence includes the following combinations:

- Travelers and entry records — connects traveler information to recorded entry activity.
- Vehicles and travelers — displays vehicle records with associated traveler details.

- Users and roles — displays accounts with their role classifications.
- Departments and personnel — displays personnel alongside department information.
- Personnel and checkpoint assignments — connects staff records to assignment records.
- Lanes and checkpoints — displays lane information with its checkpoint.
- Payments and fee rules — connects payment-related records to fee definitions.
- Entry types and fee rules — relates entry classifications to applicable fee-rule records.
- Vehicle types and fee rules — relates vehicle classifications to fee-rule records.
- Violations and alerts — connects violation information to related alert records.
- Alerts and checkpoints — displays alerts with checkpoint information.
- Audit logs and users — associates activity records with user accounts.
- Additional JOIN evidence covers fee rules, payments, penalties, and violations as shown in the saved JOIN_Queries screenshots.

The screenshots are stored in documentation/Screenshots/JOIN_Queries/. They serve as evidence that the group executed SQL JOIN statements against the project database. The actual SQL statement text and returned row counts should be taken from the corresponding saved SQL scripts or phpMyAdmin results if a more detailed query-by-query appendix is required.

12. Testing Procedures and Results

Testing was performed in the local development environment. The following results are supported by the saved screenshots and test outputs:

Test Area	Test Performed	Observed Result
PHP application pages	Opened the dashboard and pages for Travelers, Vehicles, Entry Records, Entry Report, Checkpoints, Fee Rules, Users, and Violations.	Screenshots document that these pages were opened in the local application.
PHP CRUD examples	Captured Create, Update, and Delete examples for Travelers, Vehicles, and Entry Records.	Screenshots are saved for each of the three operations in those modules.
Database schema	Viewed the table list and structures in phpMyAdmin, including borderstations.	Table-list and table-structure screenshots are saved.
SQL JOIN queries	Executed JOIN queries and captured results in phpMyAdmin.	Multiple JOIN result screenshots are saved in the JOIN_Queries folder.
API GET requests	Opened GET endpoints for alerts, assignments, audit logs, checkpoints, entry records, personnel, shifts, travelers, users, vehicles, and violations.	JSON responses were displayed and screenshots were saved.
API POST traveler	Sent a POST request to create a traveler record.	The API returned success and a new traveler record with ID 6.

API PUT traveler	Sent a PUT request to update traveler ID 6.	The API returned a successful update message.
API DELETE traveler	Sent a DELETE request for traveler ID 6.	The API returned a successful deletion message.

The Express server was observed running on port 3000, and the backend terminal reported a successful MySQL connection. POST, PUT, and DELETE traveler tests were executed with PowerShell Invoke-RestMethod. The GET evidence was captured from browser endpoint responses. These tests demonstrate the specific operations listed; they are not a complete security, performance, or production-readiness assessment.

Test evidence is stored in documentation/Screenshots/ and its API_Testing, JOIN_Queries, and Database subfolders where applicable.

13. GitHub Collaboration

The group project source code is maintained in the GitHub repository named group_3. Git is used to track changes to project files, while GitHub provides a shared repository for collaboration.

The repository uses a protected main branch. Changes are intended to be developed on separate branches and submitted through pull requests so that changes can be reviewed before they are merged into the main branch. This approach helps reduce accidental changes to the shared stable branch and provides a record of project contributions.

Repository: https://github.com/dunk3nn/group_3

For the final submission, the group should verify that all intended changes have been pushed and that required pull requests have been merged. This document does not claim that every member completed a pull request or code review unless the repository history confirms it.

14. Conclusion and Recommendations

The Group 3 Border Entry Payment and Personnel Tracking System demonstrates the design of a relational database and the integration of a PHP web application with a Node.js/Express REST API using a shared MySQL database. The project organizes information into modules covering travelers, vehicles, entry records, checkpoints, lanes, vehicle types, fee rules, payments, penalties, personnel, shifts, assignments, users, roles, violations, alerts, and audit logs.

The saved application screenshots, database structures, JOIN results, and API test outputs provide evidence of the implemented pages and selected database and API operations. In particular, traveler API creation, updating, and deletion were successfully demonstrated, and selected GET endpoints returned JSON responses.

Recommendations for future improvement include strengthening input validation and error handling; enforcing role-based permissions consistently; using secure password hashing and protected configuration; adding automated tests; documenting each API route and database constraint; implementing backup and recovery procedures; conducting security and performance testing; and, only if required and authorized, integrating approved external payment or border-management services.

Overall, the project provides a practical academic demonstration of database management, CRUD operations, SQL relationships, API development, and system integration.

15. References

- PHP Documentation. PHP language and runtime reference. https://www.php.net/docs.php
- Node.js Documentation. Node.js runtime documentation. https://nodejs.org/docs/
- Express Documentation. Express web framework guide. https://expressjs.com/
- MySQL Documentation. MySQL Reference Manual. https://dev.mysql.com/doc/
- Git Documentation. Git reference and user manual. https://git-scm.com/doc
- GitHub Documentation. Repository, branch, and pull-request guidance. https://docs.github.com/
- MDN Web Docs. HTML, CSS, and JavaScript reference. https://developer.mozilla.org/
- Project source code, database schema, ERD, system architecture diagram, application screenshots, SQL JOIN results, and API test evidence maintained by Group 3.

Access dates are not listed because they were not supplied in the original project materials.

16. Appendices

The following evidence files are maintained in the project's documentation directory. Include the corresponding images in the final printed or electronic appendix where required by the instructor.

- ERD.png — Entity Relationship Diagram.
- System_Architecture.png — system architecture diagram.
- Dashboard.png and module screenshots — dashboard and application pages.
- Travelers_Create.png, Travelers_Update.png, Travelers_Delete.png; corresponding Vehicles and Entry_Records screenshots — CRUD evidence.
- Database_Tables.png and table-structure screenshots — database table and schema evidence, including Borderstations_Table_Structure.png.
- JOIN_Queries folder — SQL JOIN result screenshots.
- API_Testing folder — GET, POST, PUT, and DELETE API test screenshots.
- Group3_Final_Project_Documentation.docx and Group3_Final_Project_Documentation.pdf — editable and PDF copies of this report.

Suggested appendix organization: Appendix A — ERD and architecture; Appendix B — application pages and CRUD; Appendix C — database tables and structures; Appendix D — SQL JOIN query results; Appendix E — API testing results.